Branch & Bound

Optimization Problems

- Brute force search
  - Backtracking
- Branch and Bound
- Dynamic programming
- Divide and Conquer
- Greedy

Backtrack search

- Adopts a DFS (depth-first search) strategy over the **search space**

Backtrack search

- Adopts a DFS (depth-first search) strategy over the **search space**

- Improvement?

Backtrack search

- Adopts a DFS (depth-first search) strategy over the **search space**

- Improvement?

  Violating objects can be pruned by a local check (need not wait till we need the leaf).

- Running time?

Backtrack search

- Adopts a DFS (depth-first search) strategy over the **search space**

- Improvement?

  Violating objects can be pruned by a local check (need not wait till we need the leaf).

- Running time?

  At least the number of leaves.

Improvement over Backtracking search

Can we not explore all the states from a particular state?

PRUNING: can we prune the search tree?

Do we need to explore the entire tree?

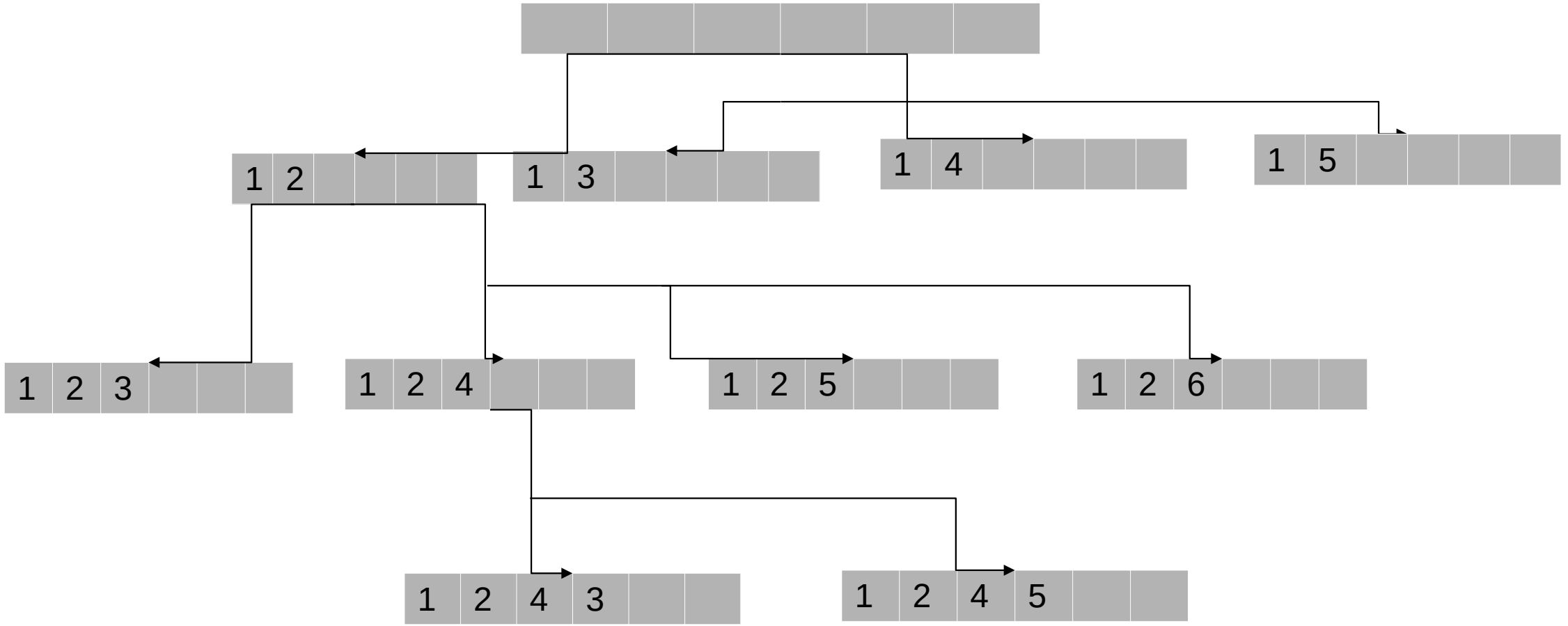
Improvement over Backtracking search

Can we not explore all the states from a particular state?

PRUNING: can we prune the search tree?

Branch and Bound: a heuristic of pruning.

Do we need to explore the entire tree?



Suppose we are given a cost for one complete object, can we use this to prune the search tree?

Branch and Bound

Let C(O) be the cost of a complete object O.

Suppose,

- $C(O_1)$ is also defined, (O_1 need not be complete)
- If O_1 can be extended to obtain O, then we know that $\mathbf{C(O_1) \leq C(O)}$

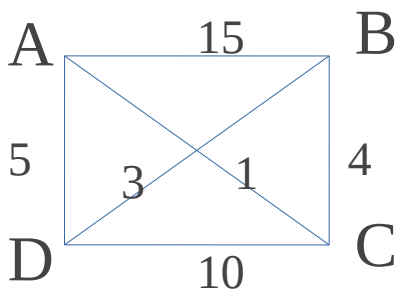
Then Branch and Bound is possible.

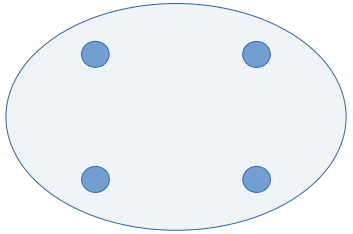
Travelling Salesperson Problem

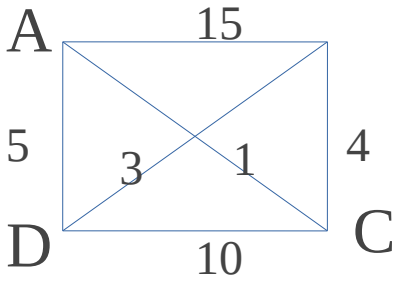
Input: A weighted graph.

Output: A “tour” through the vertices of the graph that visits every vertex exactly once.

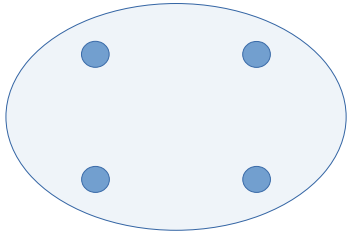
Objective function: minimize the cost of the tour in terms of distance traveled)

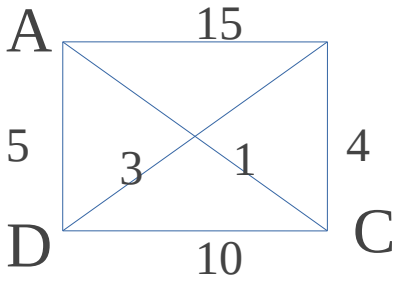




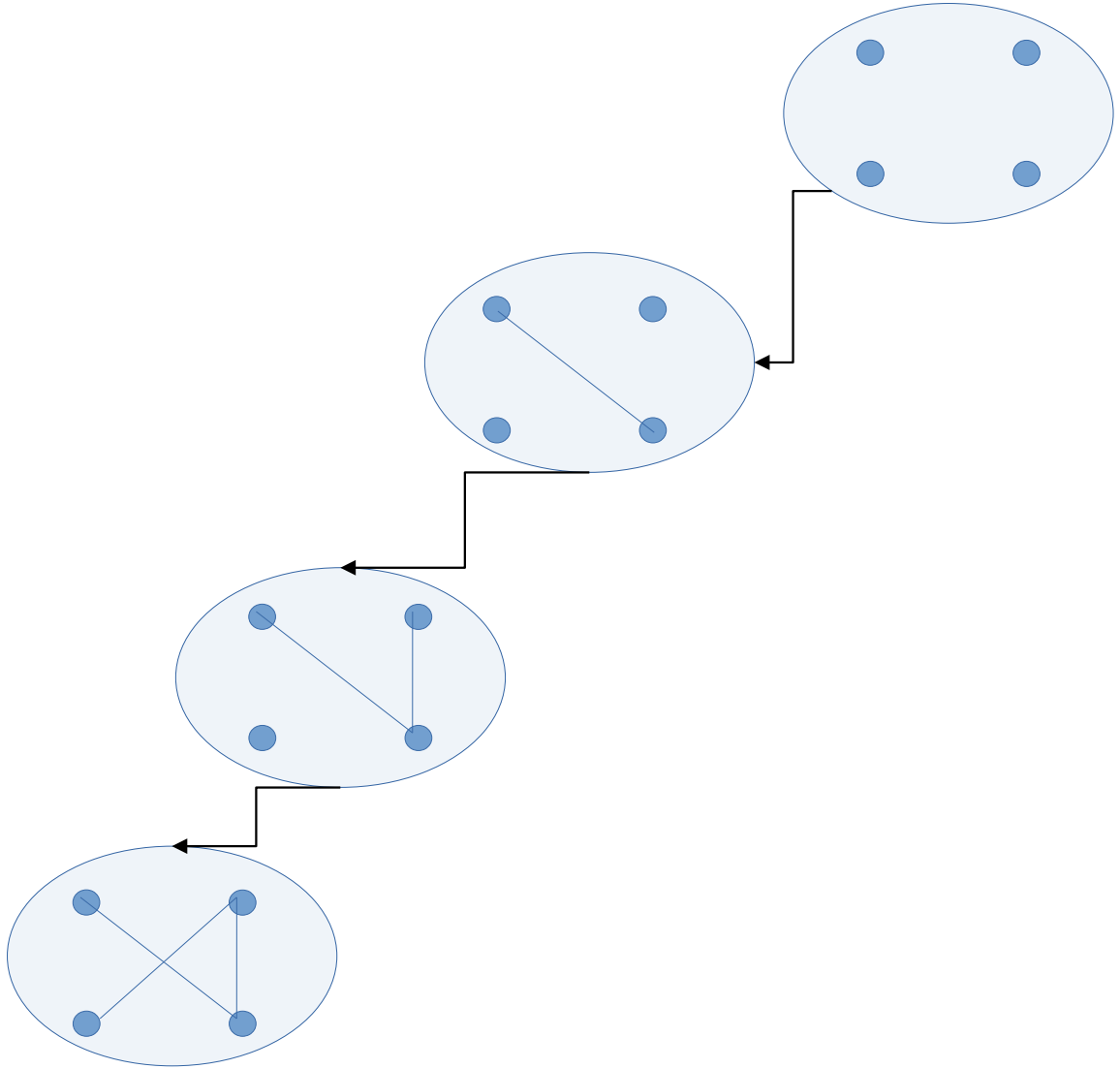


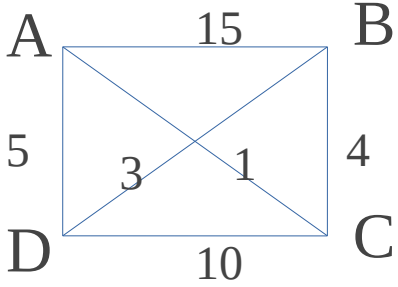
Cost: ??

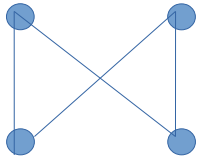




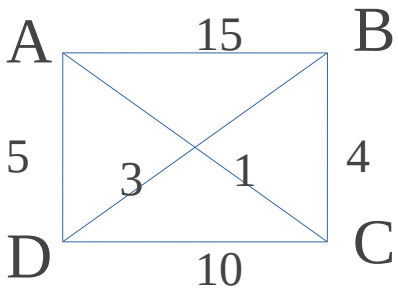
Cost: sum of edges weights of the object.

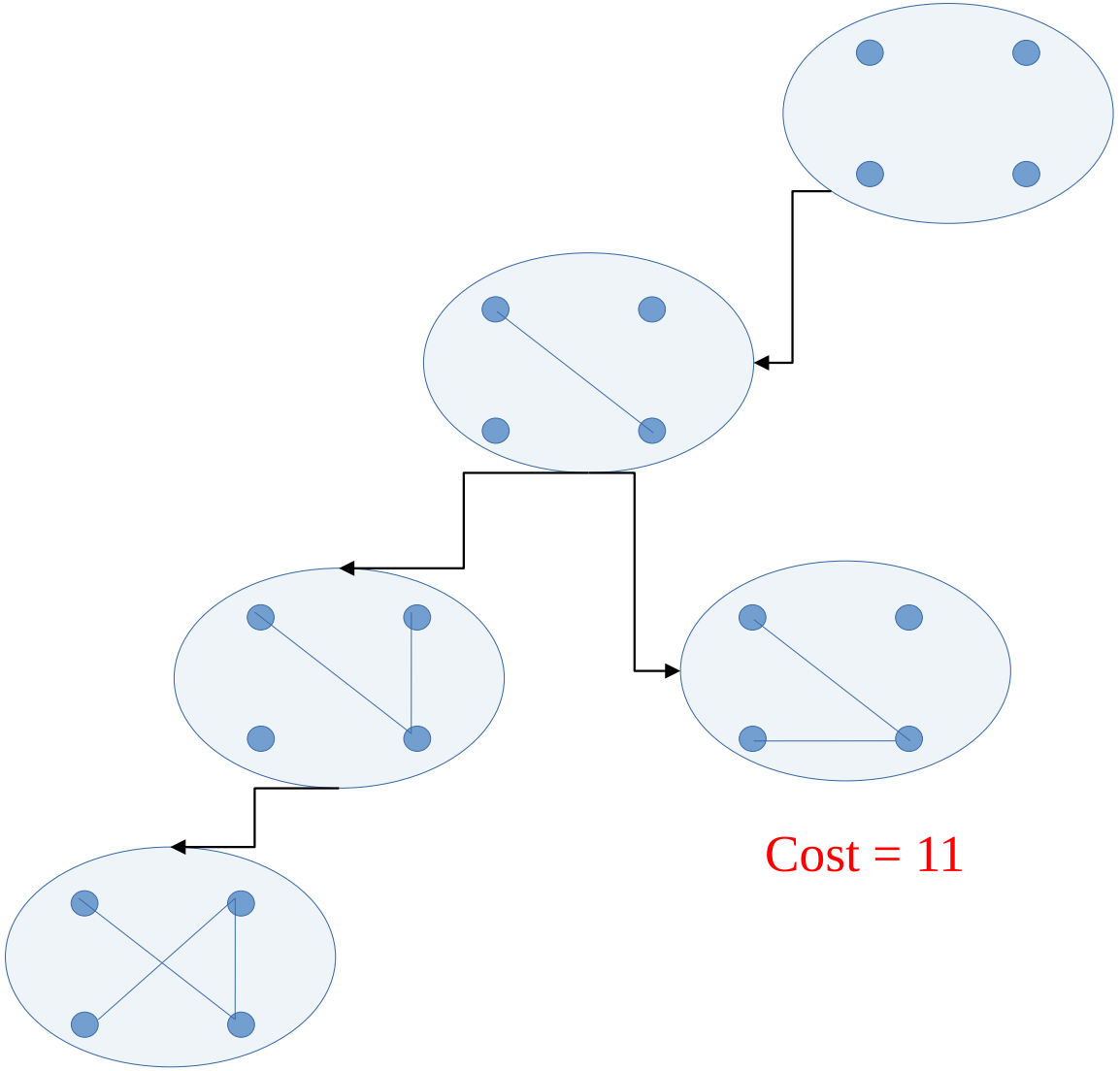




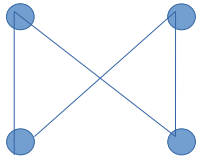


Cost = 13

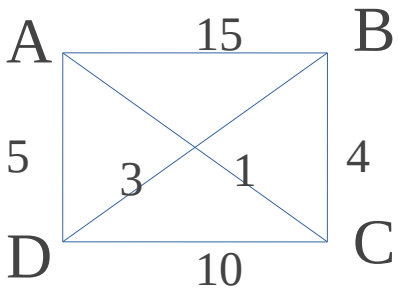


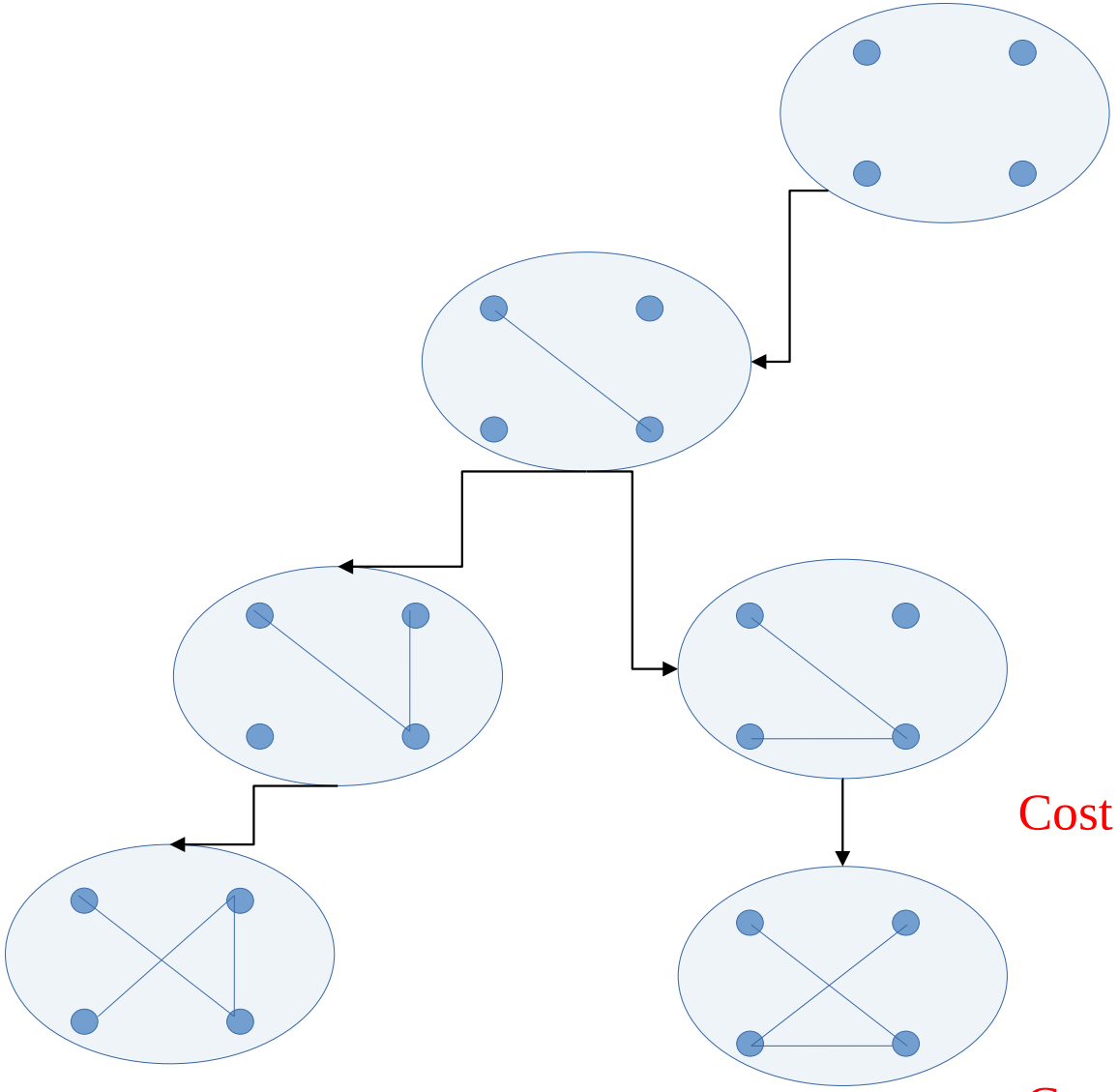


Cost = 11



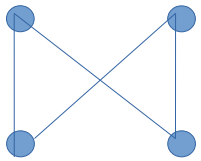
Cost = 13



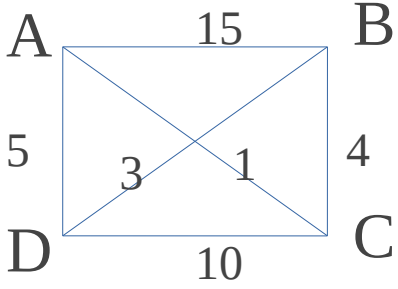


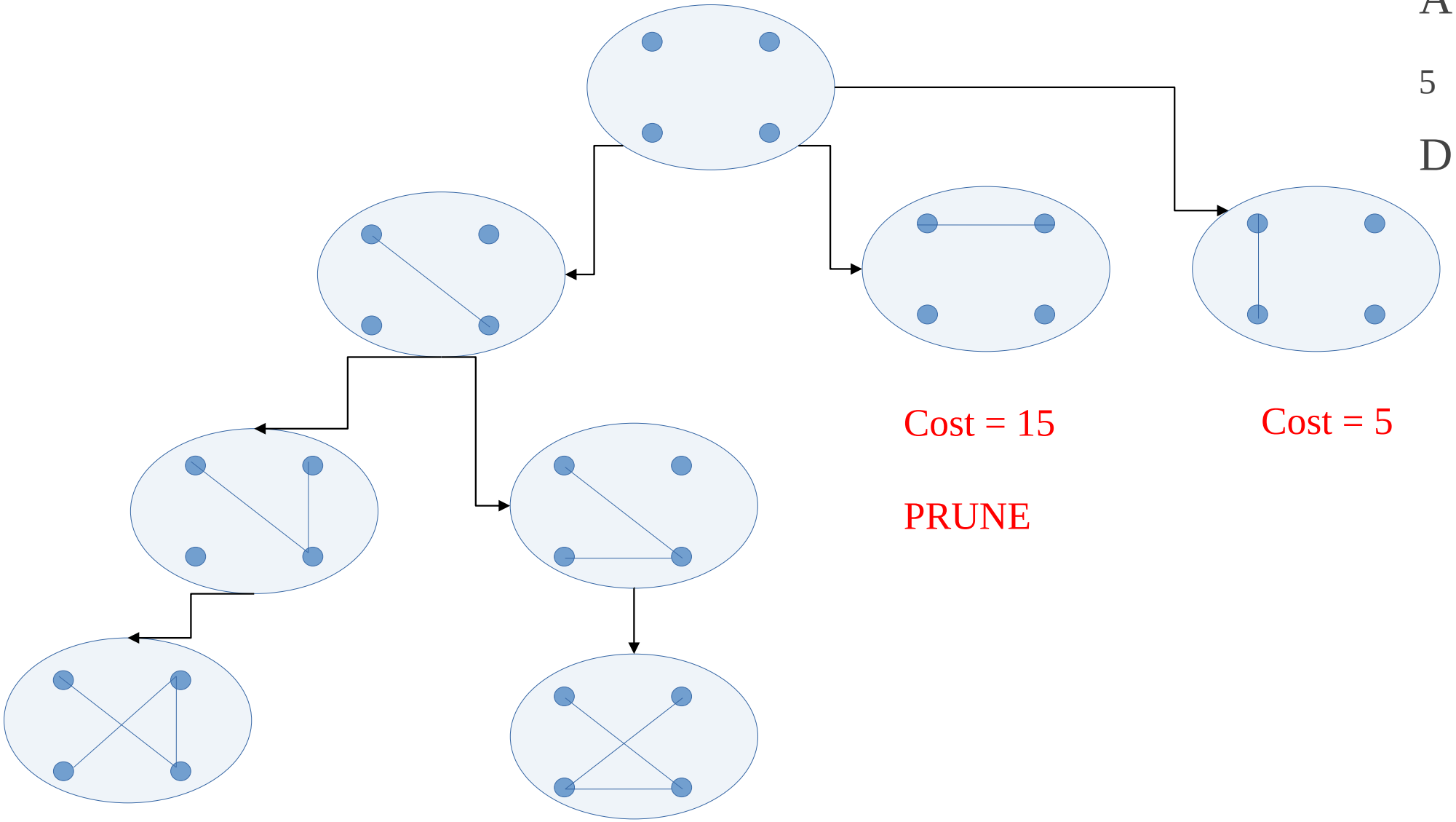
Cost = 11

Cost = 14



Cost = 13

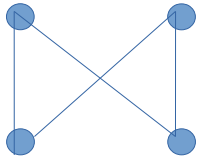




Cost = 15

PRUNE

Cost = 5



Cost = 13

Backtracking vs Branch and Bound

General method	Suitable cost function
May do more work	Pruning and therefore less work (modulo cost function head at each node)
Pruns only invalid paths	Pruns invalid and non-promising paths

Knapsack Problem

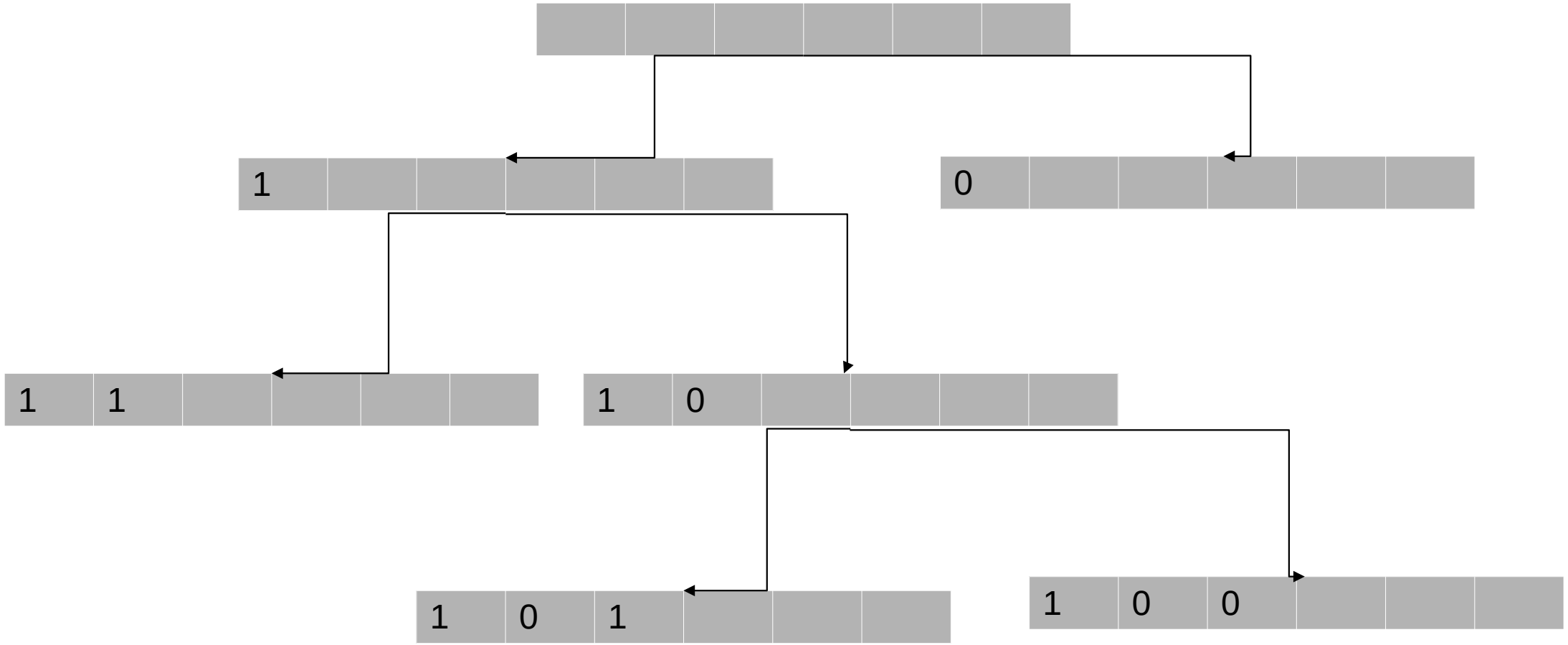
Input: A set of n items,

v1, v2, …, vn value of each of the items

W1, W2, …, Wn weights of each of the items

Capacity C

Output: Find a subset of items such that the **total value of the selected items is as large as possible** while total weight to be at most C.



#leaves? 2^n

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8

Cost: ??

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8

P={}, Y={1, 2, 3, 4}

Cost: ??

P = Set of items that we picked

Y = Set of items for which we are yet to make a decision.

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8

Cost = 0

P={}, Y={1, 2, 3, 4}

Cost: revenue generated from the items

P = Set of items that we picked

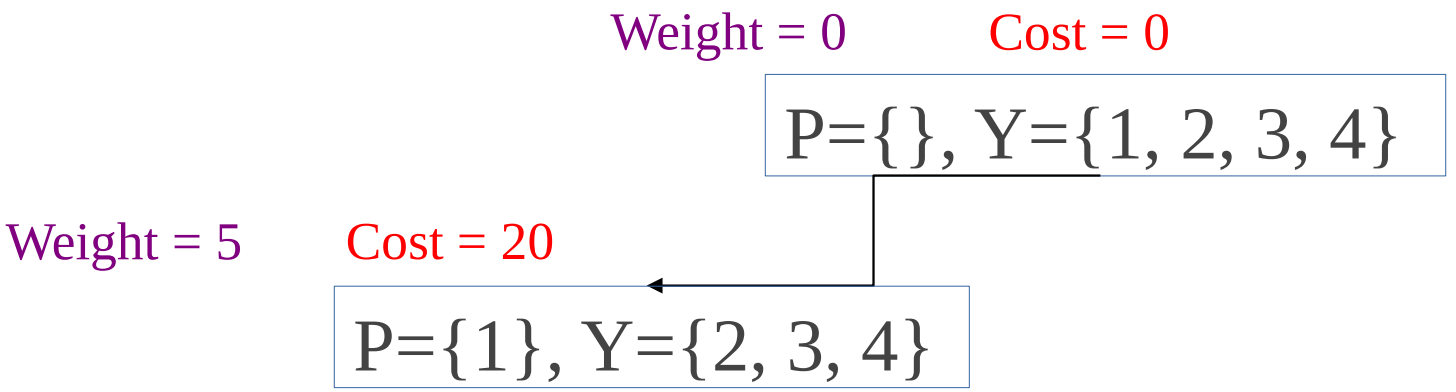
Y = Set of items for which we are yet to make a decision.

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8



Cost: revenue generated from the items

P = Set of items that we picked

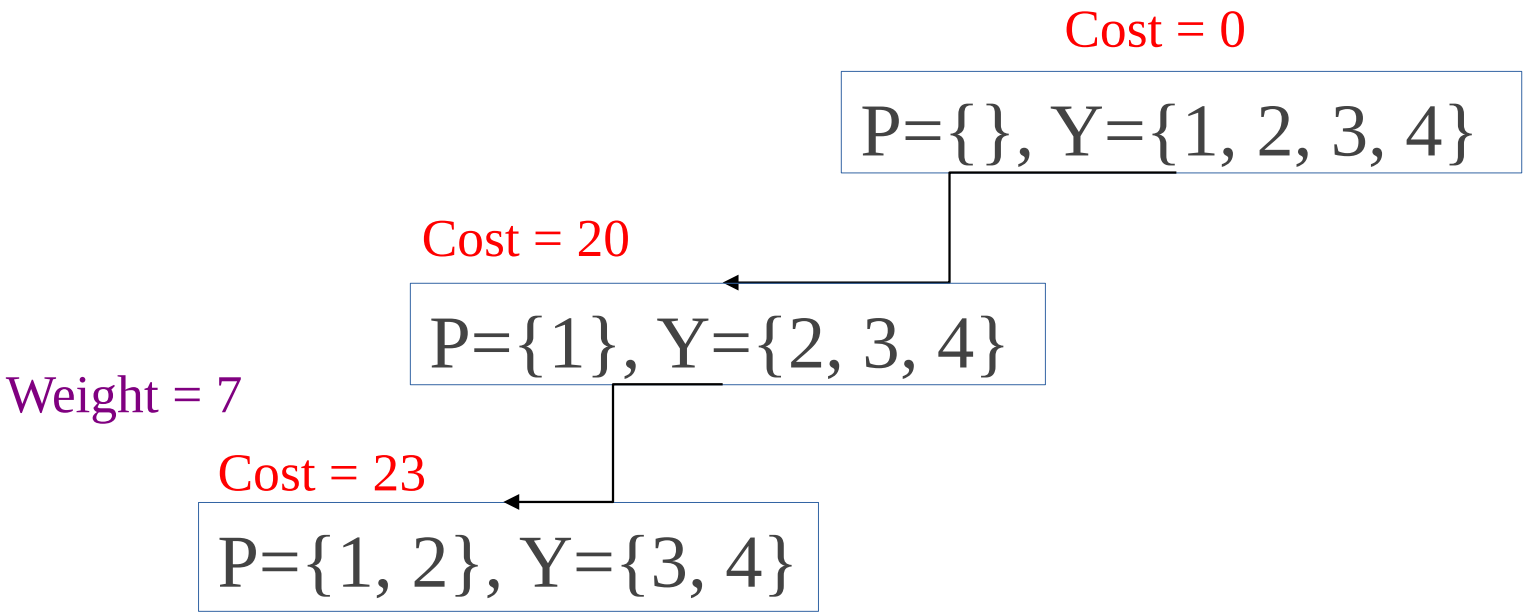
Y = Set of items for which we are yet to make a decision.

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8



Cost: revenue generated from the items

P = Set of items that we picked

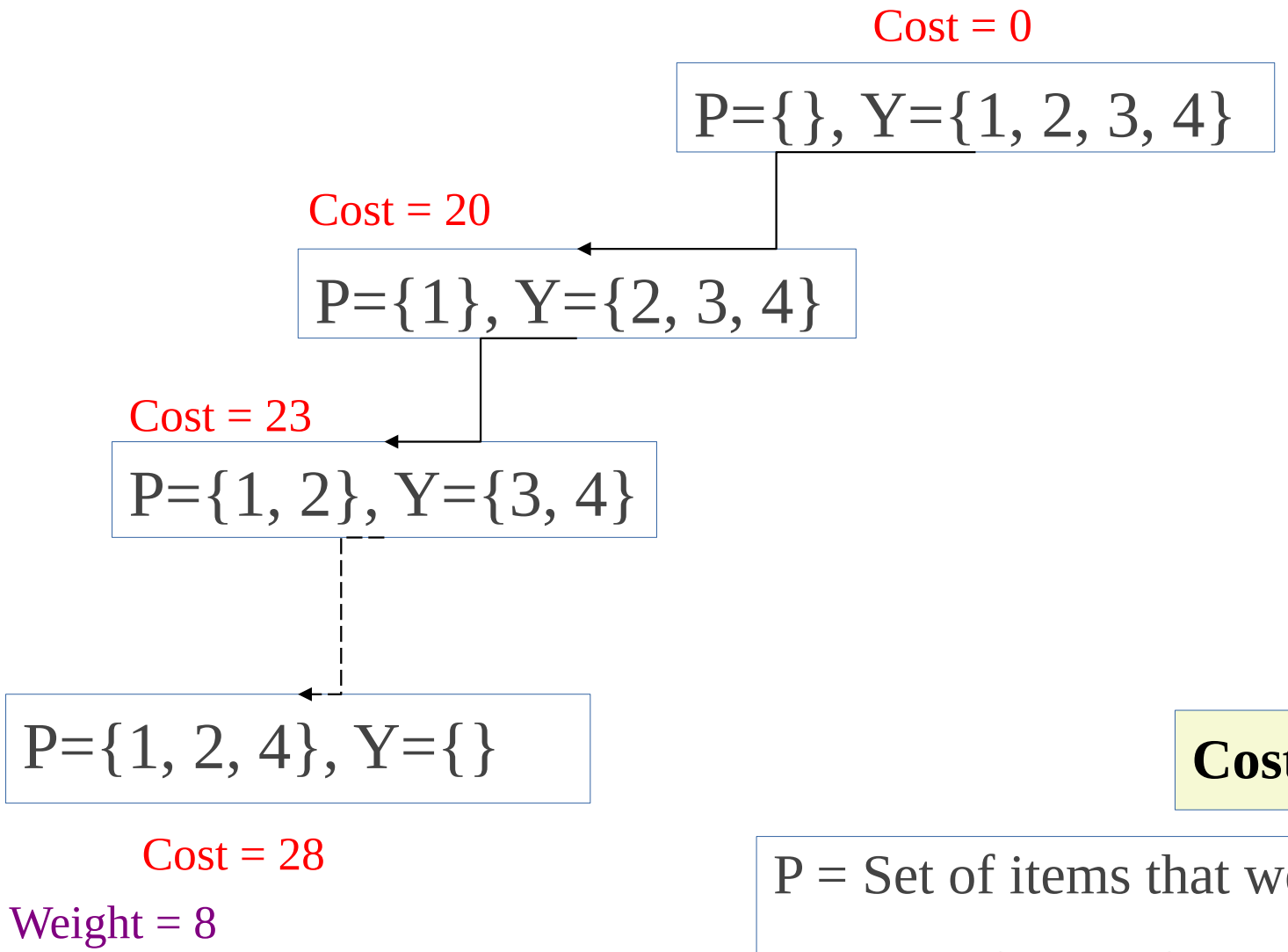
Y = Set of items for which we are yet to make a decision.

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8



Cost: revenue generated from the items

P = Set of items that we picked

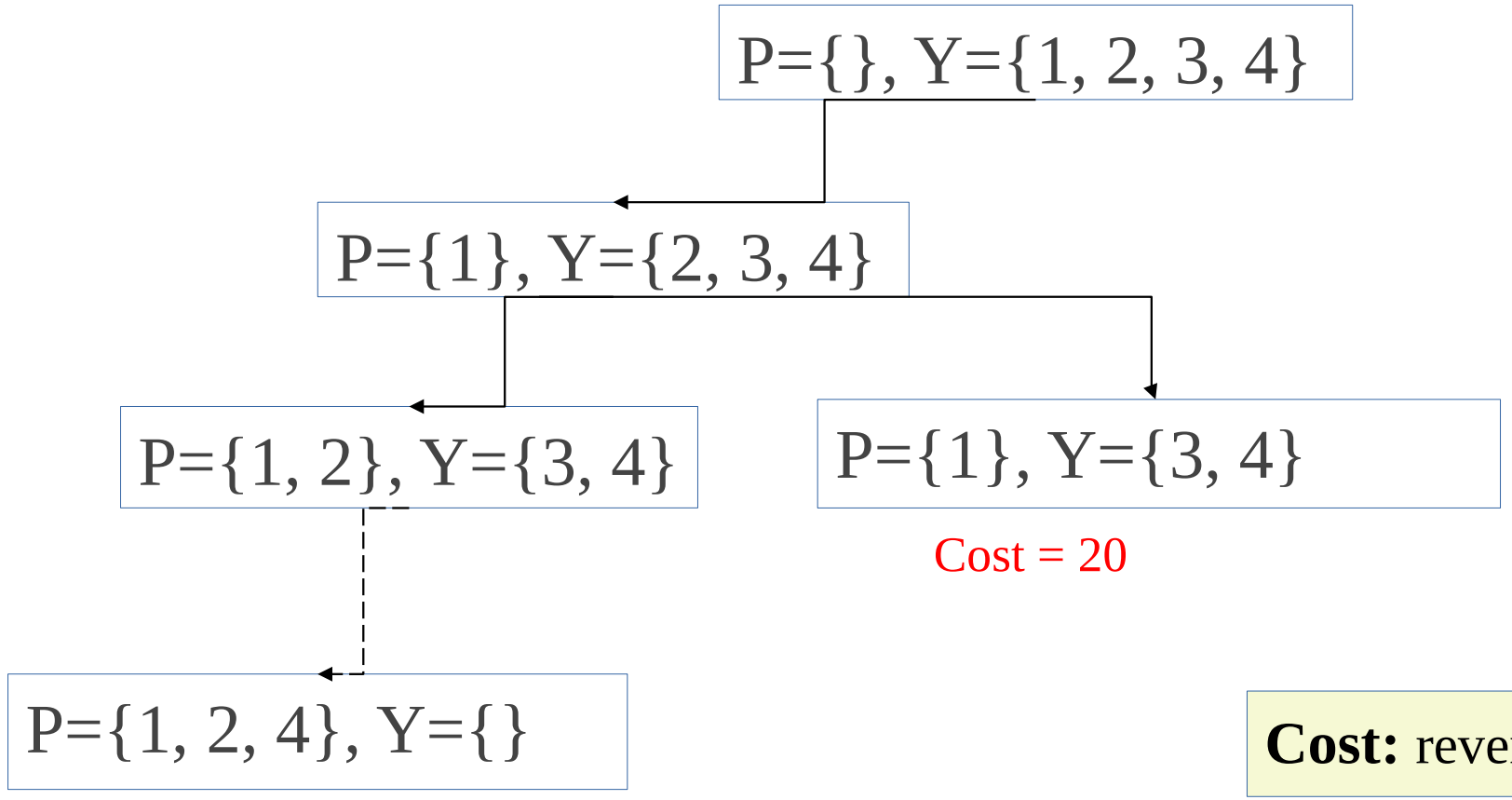
Y = Set of items for which we are yet to make a decision.

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8



Cost: revenue generated from the items

P = Set of items that we picked

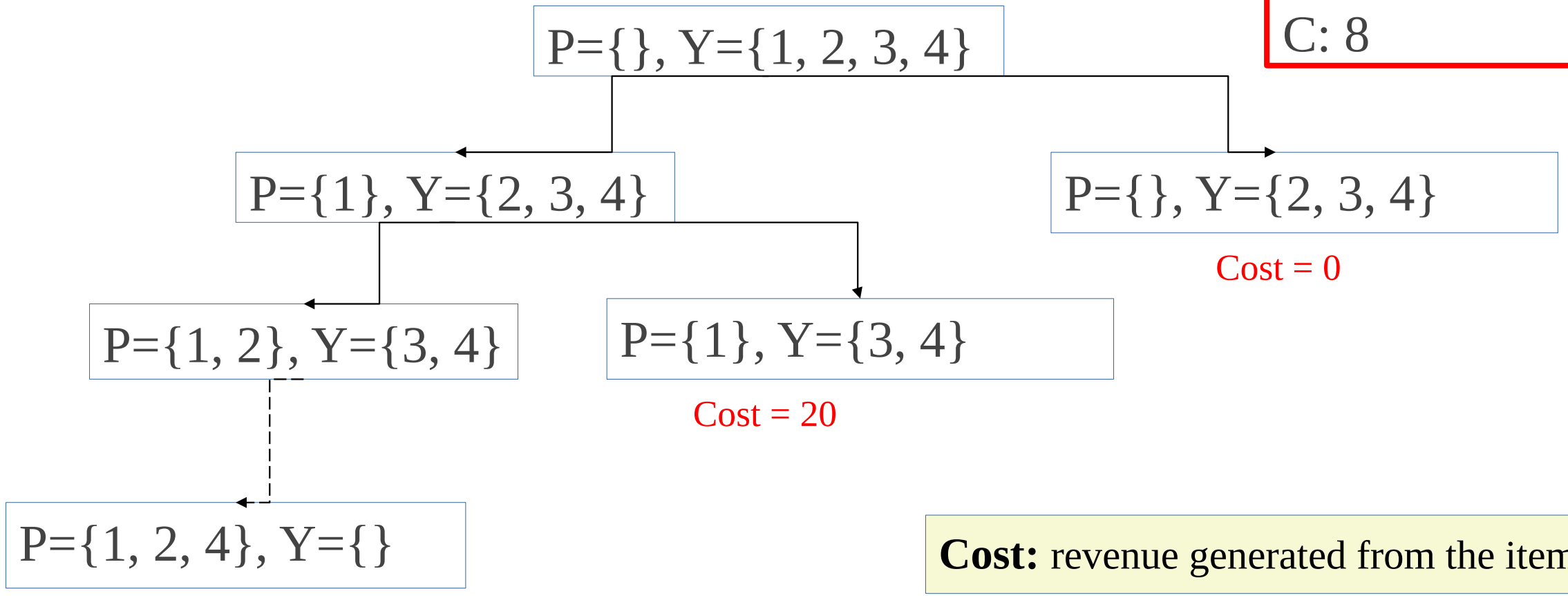
Y = Set of items for which we are yet to make a decision.

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8



Cost: revenue generated from the items

P = Set of items that we picked

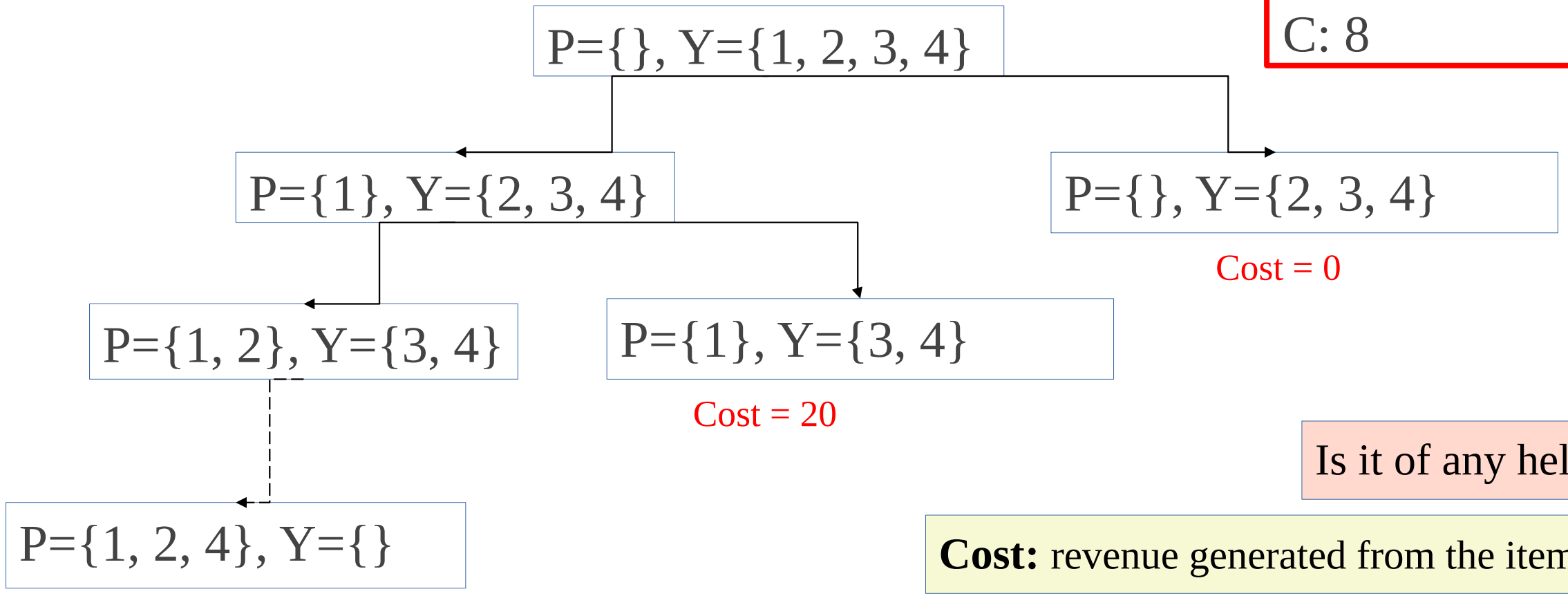
Y = Set of items for which we are yet to make a decision.

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8



Is it of any help?

Cost: revenue generated from the items

P = Set of items that we picked

Y = Set of items for which we are yet to make a decision.

Trick for the cost function

Maximize the value of the selected objects (cost= sum of selected objects)

=

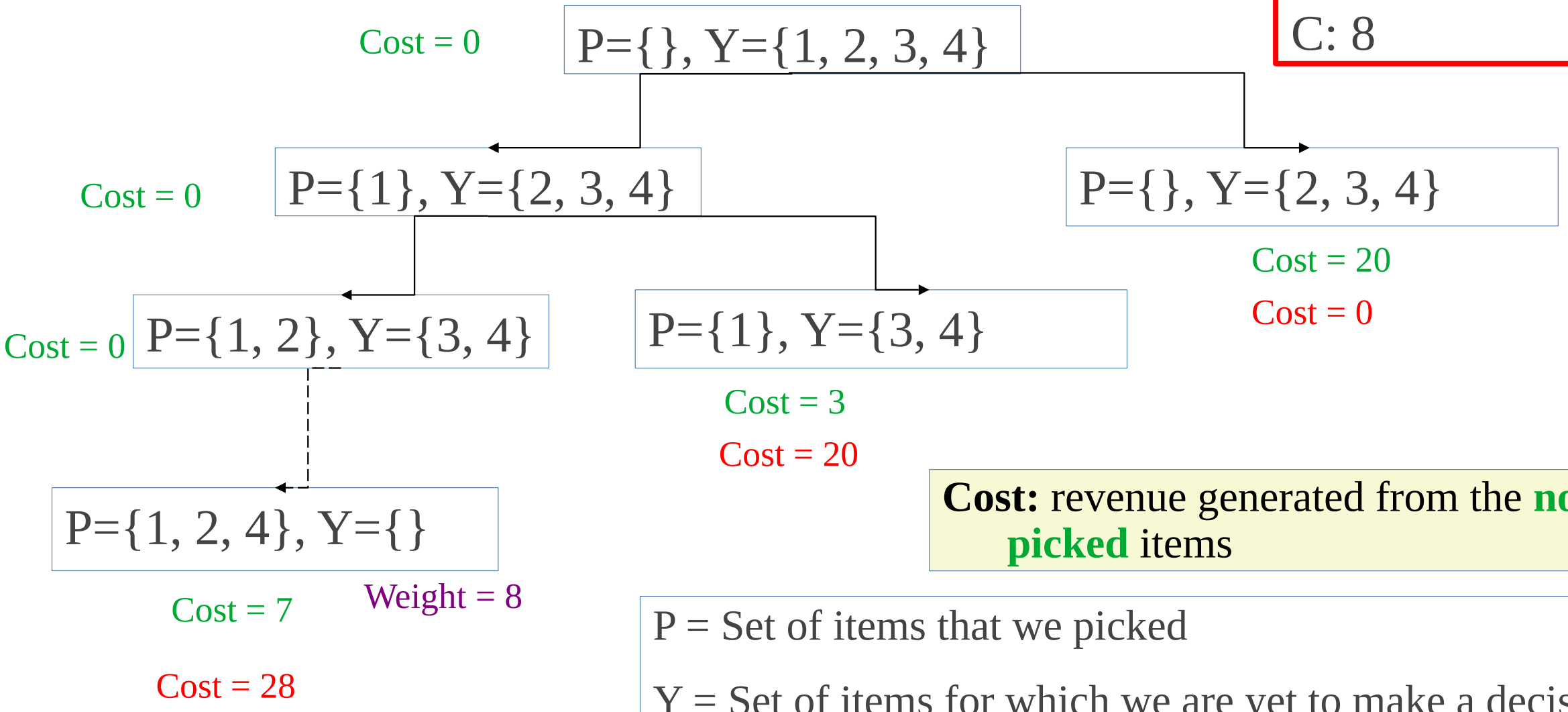
Minimize the value of the non-selected objects (cost= sum of non-selected objects)

Knapsack Problem

V: < 20, 3, 7, 5>

W: < 5, 2, 4, 1>

C: 8



Cost: revenue generated from the **non-picked** items

P = Set of items that we picked

Y = Set of items for which we are yet to make a decision.